

DATA CLEANING & PREPROCESSING REPORT

Customer Personality Analysis Dataset

Prepared for: Project Manager, AI Lab 99

Department: Machine Learning

Module: 03 — Data Cleaning and Preprocessing

Date: July 29, 2026

1. Dataset Overview

The Customer Personality Analysis dataset was received from the Data Collection and Data Understanding phase as a tab-separated file (marketing_campaign.csv) packaged inside a zip archive. It captures customer demographics, spending behaviour across product categories, campaign responses, and purchase-channel activity for a retail marketing use case.

Initial rows: 2,240

Initial columns: 29

File format: Tab-separated CSV (.csv, sep = "\t")

Numerical fields: 25 (IDs, income, recency, spend categories, purchase counts, campaign flags)

Categorical fields: 2 (Education, Marital_Status)

Date fields: 1 (Dt_Customer — stored as text on load)

An initial inspection using head(), tail(), shape, dtypes, and describe() confirmed the dataset was largely complete but carried a small set of data-quality defects typical of real-world marketing extracts: a handful of missing income values, an unconverted date column, inconsistent category labels, a small number of biologically implausible ages, and heavy-tailed distributions in the spending and campaign-response columns.

2. Identified Data Quality Issues

A systematic scan of the dataset surfaced the issues summarised in the Data Quality Assessment Table below. Severity was assigned based on the potential impact of each issue on downstream EDA and model training.

Issue	Column(s)	Description	Severity
Missing Values	Income	24 records (1.07%) had no recorded income	Medium
Incorrect Data Type	Dt_Customer	Loaded as text instead of datetime	Medium
Inconsistent Categories	Marital_Status	8 raw labels incl. non-standard "Absurd" and "YOLO"	Medium
Inconsistent Categories	Education	Casing/spacing inconsistencies (e.g. "2n Cycle")	Low
Unrealistic Values	Year_Birth	3 birth years (1893, 1899, 1900) imply ages of 114–121	High
Outliers	Income	8 extreme values, incl. one record of 666,666	High
Outliers	Mnt* spend columns	Between 35 and 248 outliers per column (right-skewed spend)	Medium
Outliers	Campaign / Response flags	Binary flags register as statistical outliers under IQR	Low
Duplicate Records	All columns	No exact duplicate rows detected	None

3. Handling Missing Values

Income was the only column with missing data: 24 of 2,240 records (1.07%) had no value.

Method Chosen

Median imputation, applied via `scikit-learn`'s `SimpleImputer(strategy='median')`.

Justification

- Income is right-skewed (max value of 666,666 vs. a median near 51,000), so the mean would be pulled upward by extreme earners; the median is robust to this skew.
- At 1.07% missing, row deletion was unnecessary and would discard otherwise-complete records.
- Mode imputation is unsuitable for a continuous variable such as income.

Post-imputation verification confirmed zero remaining missing values in Income, and by extension across the full dataset.

4. Duplicate Records

Both full-row duplicate checks (`df.duplicated()`) and an ID-uniqueness check were performed. No exact duplicate rows were found, so no records were removed at this stage.

Before	After	Removed
2,240	2,240	0

5. Correcting Data Types

Data types were reviewed column by column against expected business meaning. The key correction required was converting the customer-enrollment date from plain text to a proper datetime type; all other numeric fields were already stored correctly.

Variable	Old Type	New Type
ID	int64	int64 (retained as identifier)
Income	float64	float64 (unchanged; verified numeric)
Recency	int64	int64 (unchanged; verified numeric)
Dt_Customer	object (string)	datetime64
Education	object (string)	standardized string (title case)
Marital_Status	object (string)	standardized string (title case, merged categories)

6. Converting Date Columns

The `Dt_Customer` field was converted using the explicit source format (`%d-%m-%Y`) with `pd.to_datetime(..., errors='coerce')`, so that any unparseable value would surface as a null rather than fail silently.

Invalid / unparseable dates found: 0

Resulting dtype: datetime64[us]

Year, Month, and Day components were then extracted into separate columns (Year, Month, Day) to support later time-based feature engineering and cohort analysis, while the underlying datetime column was retained for direct date arithmetic.

7. Standardizing Categorical Variables

Both categorical columns were stripped of leading/trailing whitespace and converted to a consistent title case. Marital_Status additionally required semantic merging: "Together" was consolidated into "Married" to reflect cohabiting-partner status, while the non-standard placeholder values "Absurd" and "YOLO" — evidently data-entry artifacts rather than genuine marital statuses — were mapped to "Single".

Variable	Before	After
Marital_Status	Single, single, SINGLE, Together, Married, Divorced, Widow, Alone, Absurd, YOLO (8 raw levels)	Single, Married, Divorced, Widow, Alone (5 levels)
Education	Graduation, PhD, Master, Basic, 2n Cycle (mixed spacing/case)	Graduation, Phd, Master, Basic, 2N Cycle (title-cased, trimmed)

8. Detecting Unrealistic Customer Ages

Age was derived as (reference year – Year_Birth), using 2014 as the reference year to align with the dataset's known enrollment window. Records were flagged as unrealistic where age exceeded 100 years or fell below 18.

Unrealistic records identified: 3 (all extreme-old; no negative or sub-18 ages present)

Customer ID	Year_Birth	Derived Age
7829	1900	114
11004	1893	121
1150	1899	115

These three records (birth years 1893, 1899, and 1900) were treated as likely data-entry errors rather than genuine centenarian customers. Rather than deleting them outright and losing their otherwise-valid purchase history, they were carried into the outlier-handling stage, where the reusable preprocessing script applies an 18–100 age filter to exclude them from the final modeling-ready dataset.

9. Detecting Income and Spending Outliers

All numeric columns were screened using the IQR method (values beyond $Q1 - 1.5 \times IQR$ or $Q3 + 1.5 \times IQR$), supported by a horizontal boxplot across all numeric features for visual triage.

Column	Outliers (IQR method)
Year_Birth / Age	3

Column	Outliers (IQR method)
Income	8
MntWines	35
MntFruits	227
MntMeatProducts	175
MntFishProducts	223
MntSweetProducts	248
MntGoldProds	207
NumDealsPurchases	86
NumWebPurchases	4
NumCatalogPurchases	23
NumWebVisitsMonth	8
AcceptedCmp1–5 (campaign flags)	144–167 each
Complain	21
Response	334

The spend columns (Mnt*) and the binary campaign/response flags account for the largest outlier counts. For the binary flags this is an artifact of IQR being applied to 0/1 data rather than a genuine data-quality problem; for the spend columns it reflects a naturally right-skewed, high-value customer segment rather than erroneous entries.

10. Handling Outliers

A single, consistent Winsorization (IQR capping) pass was applied across all continuous numeric columns: values below the lower bound were set to the lower bound and values above the upper bound were set to the upper bound. This approach was chosen over removal because dropping every flagged row would have discarded a large share of the dataset (several columns had 150+ flagged rows), and over log transformation because capping preserves the original unit scale for business interpretation.

Variable	Method	Reason
Income	Cap (Winsorization)	Preserves the record while limiting the influence of extreme earners (e.g. 666,666).
Year_Birth / Age	Cap (Winsorization), reinforced by 18–100 filter in script	Removes implausible centenarian ages while retaining the record's other attributes where feasible.
Mnt* spend columns	Cap (Winsorization)	High spenders are a legitimate, business-relevant segment; capping controls leverage without discarding them.
Purchase/visit counts	Cap (Winsorization)	Consistent treatment with other numeric features; avoids introducing missing data.

Variable	Method	Reason
Binary flags (AcceptedCmp1–5, Complain, Response)	Keep (no transformation)	0/1 flags are not true outliers; IQR flags them only due to their binary distribution.

11. Validating the Cleaned Dataset

A final validation pass re-ran every check performed earlier in the workflow against the fully processed dataset to confirm each issue had been resolved.

Validation Check	Status	Evidence
Missing Values	Pass	0 nulls remain across all 33 columns
Duplicates	Pass	0 duplicate rows
Data Types	Pass	Dt_Customer is datetime64; numeric fields verified
Dates	Pass	0 invalid/unparseable dates
Categories	Pass	Education (5) and Marital_Status (5) standardized
Outliers	Pass	All numeric columns capped via IQR Winsorization

12. Saving the Processed Dataset

The cleaned dataset was exported to `customer_personality_cleaned.csv` inside a dedicated `03_Cleaned_Data/` directory, keeping raw and processed data physically separated for traceability.

13. Reusable Preprocessing Script

All steps above were consolidated into a single reusable function, `preprocess_data(path)`, so the pipeline can be re-run end-to-end on any future extract of the same schema. The function performs, in order: file load, median imputation of Income, duplicate removal, explicit-format date conversion, categorical trimming/casing and category merging, Age derivation, and an 18–100 plausible-age filter. This script is the recommended entry point for future EDA or modeling notebooks rather than repeating the exploratory steps manually.

14. Before-and-After Comparison

Metric	Before Cleaning	After Cleaning
Rows	2,240	2,240
Columns	29	33 (Age, Year, Month, Day added)
Missing Values	24 (Income only)	0
Duplicate Records	0	0
Invalid Data Types	1 (Dt_Customer as text)	0

Metric	Before Cleaning	After Cleaning
Outliers (flagged by IQR)	Present across 20+ columns	Capped to IQR bounds across all continuous columns
Date Errors	N/A (unconverted)	0

The cleaning process left the dataset's row count fully intact — no records were dropped — while resolving every identified quality issue. Four engineered columns (Age, Year, Month, Day) were added to support downstream segmentation and cohort analysis. The dataset now has zero missing values, zero duplicates, correctly typed dates, standardized categories, and numeric ranges bounded to sensible IQR limits, making it suitable for exploratory data analysis and model training.

15. Methodology, Challenges, and Recommendations

Cleaning Methodology

The workflow followed a linear, auditable sequence: (1) load and inspect, (2) catalogue every quality issue before changing anything, (3) impute missing values, (4) de-duplicate, (5) correct data types, (6) convert dates, (7) standardize categories, (8) flag unrealistic ages, (9–10) detect and cap outliers, (11) validate, and (12–13) export and encapsulate the pipeline in a reusable script. Keeping detection and correction as separate stages made it possible to justify each transformation against a concrete, quantified issue rather than applying blanket fixes.

Challenges Encountered

- The raw date format (DD-MM-YYYY) was not automatically inferred by pandas and required an explicit format string to avoid silent misparsing.
- Marital_Status contained placeholder/joke values ("Absurd", "YOLO") that required manual, judgment-based mapping rather than a purely programmatic rule.
- Applying the IQR method uniformly across both continuous spend columns and binary campaign flags produced a large outlier count for the flags that is not a genuine data-quality problem — this required interpretation rather than automatic action.
- Balancing outlier treatment for Income and the Mnt* columns: capping needed to control extreme values without erasing the legitimate high-value customer segment that is often central to personality/segmentation analysis.

Final Dataset Quality

The final dataset (2,240 rows × 33 columns) is complete, de-duplicated, correctly typed, and free of unrealistic ages or unbounded outliers, with standardized categorical levels throughout. It is considered ready for exploratory data analysis and machine learning.

Recommendations for Future Analysis

- Re-derive Age using a dynamic reference year (or the true "current date" at analysis time) rather than a hard-coded 2014 if the pipeline is reused on newer data.

- Consider excluding binary campaign/response flags from generic IQR-based outlier routines, since IQR is not a meaningful test for 0/1 data.
- Revisit the Winsorization bounds for Income specifically, given its business importance for segmentation — a domain-informed cap (e.g. a fixed percentile or known survey range) may be preferable to a purely statistical IQR bound.
- Add automated unit tests around the `preprocess_data()` function so future dataset refreshes are validated against the same rules documented in Section 11.